

```
// === App Shell (js/app.js) ===
```

```
/**
 * vÊfz\ n8b T -Æ - App Shell
 * @author ch2077
 * @version 3.2.0
 * @date 2025-05-10 etT 21k>n8b
 *
 * n8b hFgŸ j!wWS gŸg„Ÿ
 * ~ßN u T}Thg ý in
```

```
desc: '~IQxbSx wwÿ yûR`c!g•Sİ_9\ t m^-db@g x wwÿ ',ð
tags: ['~IQx', '^wg:'],ð
      module: BreakoutGameð
    },ð
  {ð
    id: 'reaction',ð
    name: 'SÍ^"mK<Ö',ð
    icon: '&i',ð
    desc:
```

```
{
  id: 'whack',
  name: 'bSw0Ÿ ',
  icon: '0=Ü9',
  desc: '30y0-PeöbSw0Ÿ Ÿ w<u%bK_ë•ŠY •ŠY}Ÿ ',
  tags: ['Ší^"', '0 •ò'],
  module: WhackGame
},
{id:
```

```
startGame() {  
    if (!this.currentGame) return;  
    const area = document.getElementById('gameArea');  
    area.innerHTML = '';  
    this.  
}
```

<script src="js/games/bubble.js"></script>ð

<script src="js/games/invaders.js"></script>ð

<script src="js/games/wordle.js"></script>ð

<script

```
});  
    },  
  
    buildLevel() {  
        const cfg = this.levelConfigs[this.level - 1];  
        const cols = cfg.cols, rows = cfg.rows;  
        const bw =
```

```
for (const brick of this.bricks) {  
    if (!brick.alive) continue;  
    if (b.x + b.r > brick.x && b.x - b.r < brick.x + brick.w
```

```
    },  
    destroy() {  
        this.running = false;  
        clearInterval(this.loop);  
        document.removeEventListener('keydown', this.handleKey);
```



```
this.cellSize = this.canvas.width / this.cols;
```

```
this.radius = this.cellSize * 0.42;
```

```
this.shooter = { x: this.canvas.width / 2,
```

```
return;
```

```
}
```

```
for (let i = 0; i < this.bubbles.length; i++) {
```

```
const b = this.bubbles[i];
```

```
const dx = cb.x - b.x;
```

```
    },  
    findMatches(bubble) {  
        const visited = new Set(), matches = [], stack = [bubble];  
        while (stack.length) {  
            const b = sta
```

```
    },  
    loop() {  
        if (!this.running) return;  
        this.update(); this.draw();  
        this.animFrame = requestAnimationFrame(() => this.loop());  
    }  
}
```



```
const now = Date.now();
```

```
if (now - this.lastShot < this.shotCooldown) return;
```

```
this.lastShot = now;
```

```
this.bullets.push({ x: t
```

```
const colors = ['#54a0ff', '#ffa726', '#ff6b6b'];
```

```
ctx.fillStyle = colors[a.type];
```

```
ctx.fillRect(a.x - a.w/2, a.y - a.h/2,
```



```
d.innerHTML = `
```

```
<button class="dpad-btn dpad-up" id="dpUp">%²</button>
```

```
<button class="dpad-btn dpad-left" id="dpLeft">%À</b
```

? `<h3>0<ß% `mUœ• Qsÿ </h3><p>• •ÇN†Qh•è \${this.maxLevel} Qsÿ g ~È_R : \${this.score}</p><butt
: `<h3>n8b ~Óg_ÿ </h3><p>{, \${this.level}/\${this.maxLevel} Qs | _

```
    },  
    doMove(dir) {  
        let moved = false;  
        const old = this.grid.map(r => [...r]);  
        if (dir === 'up') moved = this.moveUp();  
        el
```

```
showResult(msg) {  
    document.removeEventListener('keydown', this._handleKey);  
    const div = document.createElement('div');  
    div.
```

```
['tetLeft', 'tetRight', 'tetRotate', 'tetDrop'].forEach(id => {
```

```
const el = document.getElementById(id);
```

```
if (!el) return
```

```
    if (!this.collides(this.piece.shape, np)) this.piecePos = np;
    },
    hardDrop() {
        while (!this.collides(this.piece.shape, { x: th
```

```
let placed = 0;
```

```
while (placed < this.mines) {
```

```
const r = Math.floor(Math.random() * this.rows);
```

```
const c = Math.floor(Ma
```

```
    }  
    }  
  
    if (this.checkWin()) {  
        this.showResult(true);  
    }  
},  
  
toggleFlag(r, c, cell) {  
    if (this.gameOver || this.isCellOccupied(r, c)) {  
        return;  
    }  
    if (this.isCellOccupied(r, c)) {  
        return;  
    }  
    this.grid[r][c] = cell;  
    this.isCellOccupied(r, c) = true;  
    this.checkWin();  
    this.checkDraw();  
}
```



```
running: false,Ø
```

```
swapping: false,Ø
```

```
Ø
```

```
init(container, statsCb) {Ø
```

```
this.container = container;Ø
```

```
this.statsCb = statsCb;Ø
```

```
this.score
```

```
if (tr >= 0 && tr < this.size && tc >= 0 && tc < this.size) {
```

```
this.selectCell(tr, tc);
```

```
}
```

```
this._touchStar
```

```
    for (const m of newMatches) matches.push(m);  
  }  
    
  this.refreshDisplay();  
  this.updateStats();  
  this.swapping = false;  
}
```

```
document.getElementById('replayBtn').addEventListener('click', () => {  
    this.init(this.container, this.statsCb);  
});  
},  
de
```